Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Applied Cryptography

ENCS4320

HW 2

Partners:

Maysa Khanfar	1221623	Section 1
Lara Foqaha	1220071	Section 3

Date:

January, 2026

Abstract

This assignment presents the design and implementation of a secure client–server communication system that applies both symmetric and asymmetric cryptographic techniques to protect data transmitted over a TCP network. In Phase 1, confidentiality is achieved using the Advanced Encryption Standard (AES) in Cipher Block Chaining (CBC) mode with PKCS#7 padding. A symmetric encryption key is deterministically derived from the student identifier, while a fresh initialization vector (IV) is generated for each message to ensure semantic security. The IV is transmitted alongside the ciphertext, enabling correct decryption at the receiver. In Phase 2, public-key cryptography is introduced to address secure key distribution and authentication, eliminating the need for prior shared secrets. The system follows a modular design that separates cryptographic operations from networking and protocol logic. Security is validated by successful decryption at the receiver and by verifying that plaintext data is not visible in captured network traffic.

Table of Contents

Abstract 2
Table of Figures 4
Phase I: Establishing Encrypted Communication 5
Phase II: Key Exchange and Authentication 9
Security Guarantees 14
Conclusion 16

Table of Figures

Figure 1: Server listening and connecting to client 5
Figure 2: Key deriving code 5
Figure 3: PKCS7 padding code..... 5
Figure 4: CBC encryption code 6
Figure 5: IV generation code 6
Figure 6: Client encryption 7
Figure 7: Encrypted traffic using wireshark 8
Figure 8: Client-server handshake 9
Figure 9: Encrypted data using session key 10
Figure 10: CLIENT_HELLO message sent by client..... 11
Figure 11: SERVER_HELLO message sent by server..... 12
Figure 12: Encrypted message exchange 13
Figure 13: Integrity check 14
Figure 14: Authenticity check..... 15

1. Phase I: Establishing Encrypted Communication

In this phase, a secure communication channel was established between a client and a server using a TCP connection. The client encrypts user messages using AES-128 in CBC mode before transmission. The server receives the encrypted data, extracts the initialization vector (IV), decrypts the ciphertext using the same AES key, and recovers the original plaintext. This phase focuses on confidentiality and correct encryption/decryption without exposing plaintext over the network.

```
[server] listening on 127.0.0.1:5005
[server] connected from ('127.0.0.1', 63337)
```

Figure 1: Server listening and connecting to client

Key Generation from Student ID

The AES-128 secret key is deterministically derived from the student's ID to ensure a unique and reproducible key. We used one of our IDs which is 1220071 and imported the hash library "hashlib" to hash using the MD5 algorithm, producing a 128-bit output that is directly used as the AES key. We got the following key:

Key: a881c1ae9a114c66044323904a708f82

This approach guarantees that both the client and server independently derive the same symmetric key without explicitly exchanging it over the network.

```python
def derive_key(student_id):
    return hashlib.md5(student_id.encode()).digest()
```

Figure 2: Key deriving code

AES-CBC Encryption with PKCS#7 Padding

AES encryption is implemented in Cipher Block Chaining (CBC) mode. Since AES operates on fixed 16-byte blocks, PKCS#7 padding is applied to the plaintext to ensure its length is a multiple of the block size. Each plaintext block is then XORed with the previous ciphertext block (or IV for the first block) before encryption, providing diffusion and protecting against pattern leakage.

```python
def pkcs7_pad(data, block_size=16):
    pad_len = block_size - (len(data) % block_size)
    return data + bytes([pad_len]) * pad_len
```

Figure 3: PKCS7 padding code

```python
def cbc_encrypt(msg, key, iv):
    padded = pkcs7_pad(msg)
    round_keys = key_expansion(key)   # expanded AES round keys

    ciphertext = b""
    prev_block = iv

    for i in range(0, len(padded), 16):
        block = padded[i:i+16]
        block = xor_bytes(block, prev_block)  # CBC XOR

        # AES block encryption
        encrypted = aes_encrypt_block(block, round_keys)

        ciphertext += encrypted
        prev_block = encrypted

    return ciphertext
```

Figure 4: CBC encryption code

Initialization Vector (IV) Generation and Transmission

For every encrypted message, a fresh initialization vector (IV) is generated. The IV is 16 bytes long and is not required to be secret. To ensure uniqueness across messages, an incrementing counter is applied to a reference IV derived from the student ID. The IV is added to the ciphertext and transmitted alongside it so the receiver can correctly decrypt the message.

```python
def derive_reference_iv(student_id):
    return hashlib.sha256(student_id.encode()).digest()[:16]

def iv_from_reference(ref_iv, counter):
    # converting base IV from bytes to 128bit integer
    x = int.from_bytes(ref_iv, "big")

    # incrementing IV using counter for randomness (wraps at 2^128)
    x = (x + counter) % (1 << 128)

    # convert back to 16 byte IV
    return x.to_bytes(16, "big")
```

Figure 5: IV generation code

```
Enter message to send (or q to quit): hello

[client] sent encrypted data
Key: a881c1ae9a114c66044323904a708f82
IV: d50e180a2c417be33281bd4a00cd932e
Ciphertext: 004ac774e84562ac152d181888aa1b90
IV || Ciphertext: d50e180a2c417be33281bd4a00cd932e004ac774e84562ac152d181888aa1b90
[client] server replied: OK
====================================

Enter message to send (or q to quit): hi

[client] sent encrypted data
Key: a881c1ae9a114c66044323904a708f82
IV: d50e180a2c417be33281bd4a00cd932f
Ciphertext: 515e21323fc155c7e14a05e6bbe0c2b8
IV || Ciphertext: d50e180a2c417be33281bd4a00cd932f515e21323fc155c7e14a05e6bbe0c2b8
[client] server replied: OK
====================================
```

Figure 6: Client encryption

For each encrypted message, a unique 16-byte Initialization Vector (IV) is generated to ensure semantic security in AES-CBC mode. A reference IV is first derived deterministically from the student ID using SHA-256 for testing consistency. To guarantee IV freshness across multiple messages, the reference IV is incremented using a counter, producing a new IV for every transmission while preserving the required 128-bit size. The IV is transmitted in plaintext alongside the ciphertext (IV ‖ C), as it is not secret but required for correct decryption at the receiver. Network captures confirm that varying IVs result in different ciphertexts.

Verification of Encrypted Traffic (Wireshark)

Network traffic was captured using Wireshark to verify that no plaintext data appears on the wire. Wireshark was used to capture the TCP traffic exchanged between the client and server over the loopback interface. The capture was filtered using the application port (TCP port 5005). Inspection of the packet payload shows that the transmitted data appears as random hexadecimal values, and no readable plaintext is visible in the captured packets. This confirms that AES-CBC encryption successfully protects the confidentiality of the transmitted messages.

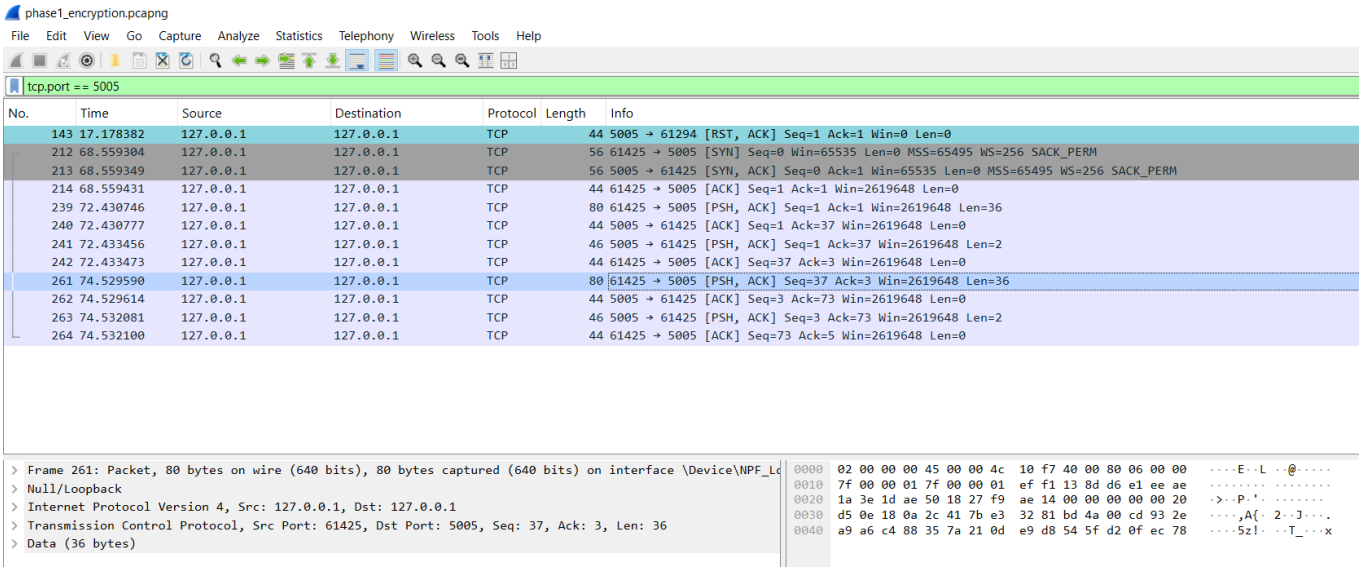


Figure 7: Encrypted traffic using wireshark

The Wireshark capture confirms that the implemented AES-CBC encryption successfully protects the confidentiality of transmitted messages. As shown in the captured packets filtered on TCP port 5005, the payload appears as random hexadecimal data rather than readable plaintext. Even when short and repeated messages are sent, the observed ciphertext differs due to the use of a fresh Initialization Vector (IV) for each transmission. This demonstrates that the encryption scheme prevents plaintext leakage over the network and achieves the intended security.

2. Phase II: Key Exchange and Authentication

In this phase, secure key exchange and mutual authentication are introduced to eliminate the need for a pre-shared symmetric key. Instead of deriving the AES key directly from a student ID, the client and server dynamically establish a shared session key using the Diffie-Hellman key exchange protocol. Public key cryptography is employed to authenticate both parties and protect the key exchange from man-in-the-middle attacks.

Before any encrypted application data is transmitted, the client and server perform a handshake protocol to verify identities and agree on a shared session key. This phase focuses on secure key establishment, authentication, and the transition to encrypted communication using session-based symmetric keys.

Handshake Protocol Overview

The handshake begins when the client establishes a TCP connection to the server and sends a CLIENT_HELLO message. This message contains the client identifier, a Diffie-Hellman public value, and a digital signature generated using the client's RSA private key. The signature ensures the authenticity of the client and protects the integrity of the Diffie-Hellman parameters.

Upon receiving the CLIENT_HELLO, the server verifies the client's signature using the corresponding RSA public key. If verification succeeds, the server responds with a SERVER_HELLO message containing its own Diffie–Hellman public value and a digital signature. The client then verifies the server's signature, completing mutual authentication.

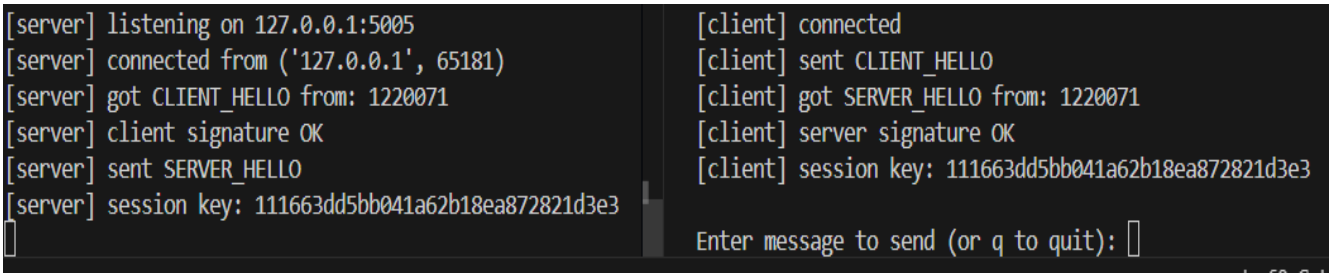


Figure 8: Client-server handshake

Authentication Infrastructure

Public key cryptography is used to authenticate both participants during the handshake phase. Each party generates an RSA-128 key pair, where the public key is associated with the corresponding Student ID and serves as the participant's identity. The private key is kept secret and is used to digitally sign protocol messages. During the handshake, the client signs its Diffie-Hellman public value using its RSA private key and sends it within the CLIENT_HELLO message. The server verifies this signature using the client's RSA public key before proceeding. Similarly, the server signs its own DiffieHellman public value in the SERVER_HELLO message, which is then verified by the client. This authentication mechanism prevents impersonation attacks, as only the legitimate holder of the private key associated with a given Student ID can generate a valid signature.

Session Key Establishment

After successful signature verification, both the client and server independently compute the shared Diffie–Hellman secret. This shared secret is then processed to derive a 128-bit session key, which is used as the AES key for all subsequent encrypted communication. The session key is never transmitted over the network and exists only locally at each endpoint. After computing the shared Diffie–Hellman secret s, both parties derive the session key using a cryptographic hash function as follows: Session_Key = SHA256(s)[0:16].

Only the first 16 bytes of the hash output are used, producing a 128-bit AES session key. This session key replaces the static key used in Phase I and is used for all subsequent encrypted communication. The session key is generated once per connection and remains constant throughout the communication session, providing confidentiality while avoiding repeated key derivation for each message. Figure 8 above shows the computed session key.

Encrypted Data Exchange Using Session Keys

Once the handshake is complete and the session key is established, the client begins transmitting encrypted application data. Messages are encrypted using AES in CBC mode with PKCS#7 padding, identical to Phase I, but using the negotiated session key instead of a static key. A fresh Initialization Vector (IV) is generated for each message and transmitted alongside the ciphertext.

The server receives the encrypted data, extracts the IV, decrypts the ciphertext using the session key, and recovers the original plaintext. Successful decryption confirms that both parties derived the same session key and that the handshake protocol completed correctly.

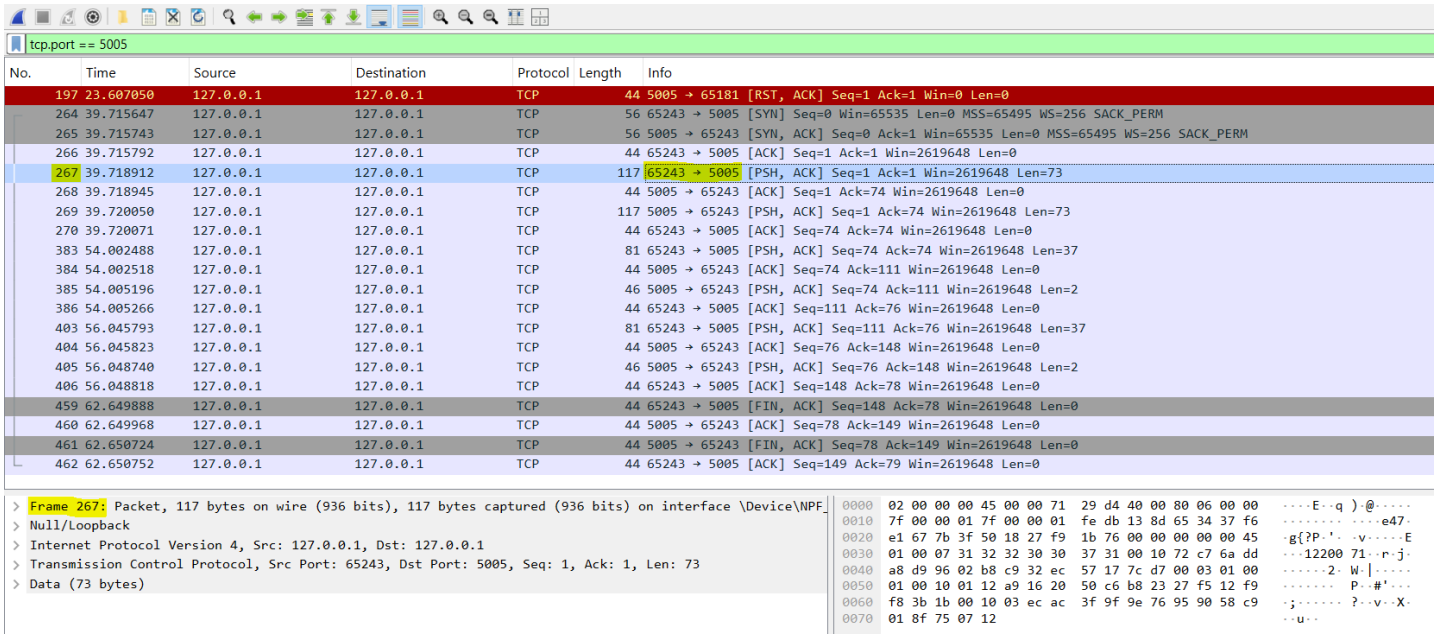
```
[client] connected
[client] sent CLIENT_HELLO
[client] got SERVER_HELLO from: 1220071
[client] server signature OK
[client] session key: 77ede0193e672574d45bae204198d347

Enter message to send (or q to quit): hello
[client] sent DATA
IV: d50e180a2c417be33281bd4a00cd932d
Ciphertext: de07735bd2d21801e47bc72ade39dab2
[client] server replied: OK

Enter message to send (or q to quit): hi
[client] sent DATA
IV: d50e180a2c417be33281bd4a00cd932e
Ciphertext: 6e611ed1a129284c3125affe8bcbcbc4
[client] server replied: OK

Enter message to send (or q to quit):
```

```
[server] listening on 127.0.0.1:5005
[server] connected from ('127.0.0.1', 65105)
[server] got CLIENT_HELLO from: 1220071
[server] client signature OK
[server] sent SERVER_HELLO
[server] session key: 77ede0193e672574d45bae204198d347
[server] decrypted: hello
[server] decrypted: hi
```

Figure 9: Encrypted data using session key

Verification Using Wireshark

- **Complete Phase II handshake (key exchange with signatures):**
  Client → Server: This is CLIENT_HELLO (handshake start)



tcp.port == 5005

No.	Time	Source	Destination	Protocol	Length	Info
197	23.607050	127.0.0.1	127.0.0.1	TCP	44	5005 → 65181 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
264	39.715647	127.0.0.1	127.0.0.1	TCP	56	65243 → 5005 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
265	39.715743	127.0.0.1	127.0.0.1	TCP	56	5005 → 65243 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
266	39.715792	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
267	39.718912	127.0.0.1	127.0.0.1	TCP	117	65243 → 5005 [PSH, ACK] Seq=1 Ack=1 Win=2619648 Len=73
268	39.718945	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=1 Ack=74 Win=2619648 Len=0
269	39.720050	127.0.0.1	127.0.0.1	TCP	117	5005 → 65243 [PSH, ACK] Seq=1 Ack=74 Win=2619648 Len=73
270	39.720071	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=74 Ack=74 Win=2619648 Len=0
383	54.002488	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=74 Ack=74 Win=2619648 Len=37
384	54.002518	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=74 Ack=111 Win=2619648 Len=0
385	54.005196	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=74 Ack=111 Win=2619648 Len=2
386	54.005266	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=111 Ack=76 Win=2619648 Len=0
403	56.045793	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=111 Ack=76 Win=2619648 Len=37
404	56.045823	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=76 Ack=148 Win=2619648 Len=0
405	56.048740	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=76 Ack=148 Win=2619648 Len=2
406	56.048818	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=148 Ack=78 Win=2619648 Len=0
459	62.649888	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [FIN, ACK] Seq=148 Ack=78 Win=2619648 Len=0
460	62.649968	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=78 Ack=149 Win=2619648 Len=0
461	62.650724	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [FIN, ACK] Seq=78 Ack=149 Win=2619648 Len=0
462	62.650752	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=149 Ack=79 Win=2619648 Len=0

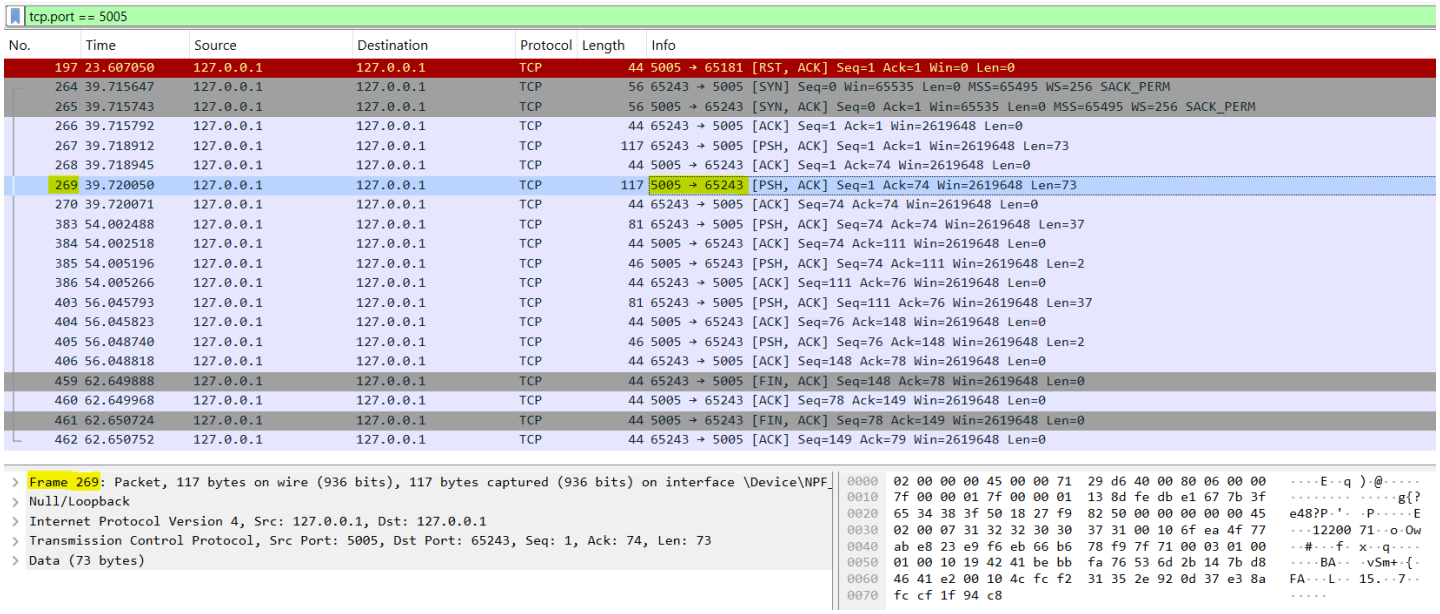
```
> Frame 267: Packet, 117 bytes on wire (936 bits), 117 bytes captured (936 bits) on interface \Device\NPF_
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 65243, Dst Port: 5005, Seq: 1, Ack: 1, Len: 73
> Data (73 bytes)

0000  02 00 00 00 45 00 00 71  29 d4 40 00 80 06 00 00   ····E··q )·@·····
0010  7f 00 00 01 7f 00 00 01  fe db 13 8d 65 34 37 f6   ........ ····e47·
0020  e1 67 7b 3f 50 18 27 f9  1b 76 00 00 00 00 00 45   ·g{?P·'· ·v·····E
0030  01 00 07 31 32 32 30 30  37 31 00 10 72 c7 6a dd   ···12200 71··r·j·
0040  a8 d9 96 02 b8 c9 32 ec  57 17 7c d7 00 03 01 00   ......2· W·|·····
0050  01 00 10 01 12 a9 16 20  50 c6 b8 23 27 f5 12 f9   ·******· P··#'···
0060  f8 3b 1b 00 10 03 ec ac  3f 9f 9e 76 95 90 58 c9   ·;****** ?··v··X·
0070  01 8f 75 07 12                                     ··u··
```

Figure 10: CLIENT_HELLO message sent by client

The Wireshark capture in Figure 10 shows the beginning of the Phase II protocol, where the client initiates the secure handshake by sending the CLIENT_HELLO message to the server. This packet is Frame 267 and contains a TCP payload of 73 bytes transmitted from the client to the server on port 5005. Inspection of the packet's payload shows that the transmitted data appears as random binary values, with no readable plaintext content. This confirms that sensitive handshake information, including Diffie-Hellman public values and RSA signatures, is not transmitted in clear text.

The presence of this encrypted handshake message before any application data exchange shows that authentication and key agreement are completed before sending user messages. Wireshark also confirms that subsequent packets also contain encrypted payloads, indicating that all application data is protected using the derived session key. This validates the correct implementation of the Phase II protocol and ensures confidentiality, authentication, and resistance to eavesdropping and impersonation attacks.

Server → Client: This is SERVER_HELLO (handshake response + signature)



tcp.port == 5005

No.	Time	Source	Destination	Protocol	Length	Info
197	23.607050	127.0.0.1	127.0.0.1	TCP	44	5005 → 65181 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
264	39.715647	127.0.0.1	127.0.0.1	TCP	56	65243 → 5005 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
265	39.715743	127.0.0.1	127.0.0.1	TCP	56	5005 → 65243 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
266	39.715792	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
267	39.718912	127.0.0.1	127.0.0.1	TCP	117	65243 → 5005 [PSH, ACK] Seq=1 Ack=1 Win=2619648 Len=73
268	39.718945	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=1 Ack=74 Win=2619648 Len=0
269	39.720050	127.0.0.1	127.0.0.1	TCP	117	5005 → 65243 [PSH, ACK] Seq=1 Ack=74 Win=2619648 Len=73
270	39.720071	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=74 Ack=74 Win=2619648 Len=0
383	54.002488	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=74 Ack=74 Win=2619648 Len=37
384	54.002518	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=74 Ack=111 Win=2619648 Len=0
385	54.005196	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=74 Ack=111 Win=2619648 Len=2
386	54.005266	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=111 Ack=76 Win=2619648 Len=0
403	56.045793	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=111 Ack=76 Win=2619648 Len=37
404	56.045823	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=76 Ack=148 Win=2619648 Len=0
405	56.048740	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=76 Ack=148 Win=2619648 Len=2
406	56.048818	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=148 Ack=78 Win=2619648 Len=0
459	62.649888	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [FIN, ACK] Seq=148 Ack=78 Win=2619648 Len=0
460	62.649968	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=78 Ack=149 Win=2619648 Len=0
461	62.650724	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [FIN, ACK] Seq=78 Ack=149 Win=2619648 Len=0
462	62.650752	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=149 Ack=79 Win=2619648 Len=0

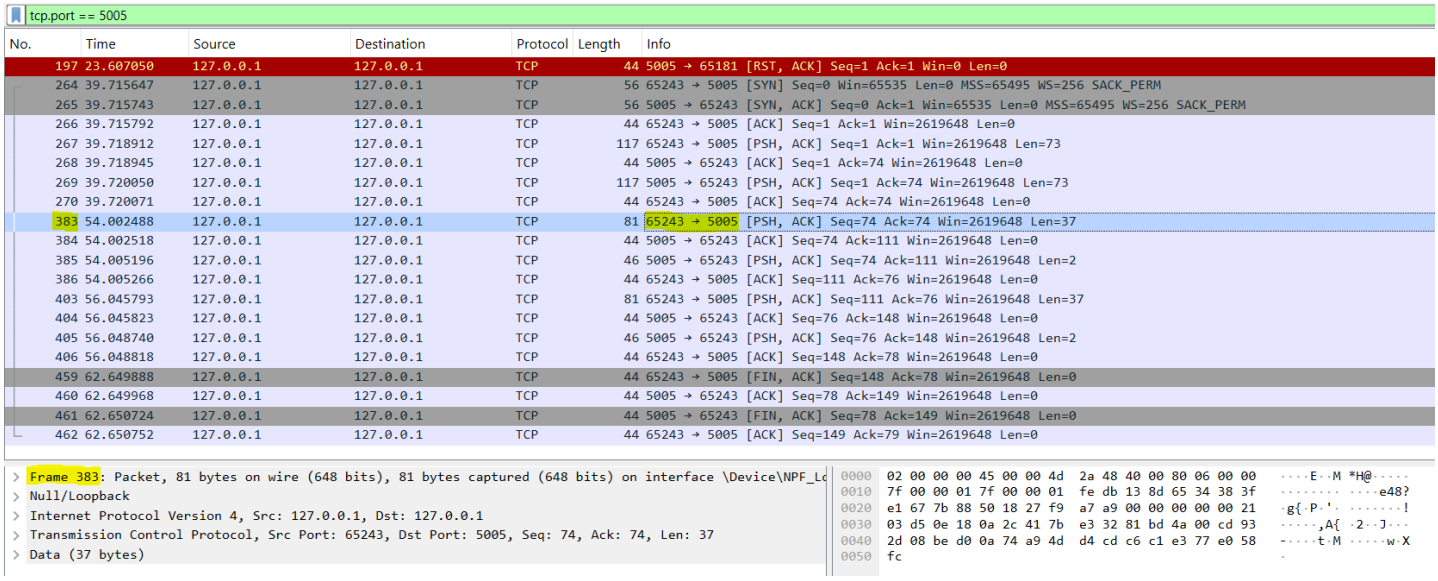
```
> Frame 269: Packet, 117 bytes on wire (936 bits), 117 bytes captured (936 bits) on interface \Device\NPF_
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 5005, Dst Port: 65243, Seq: 1, Ack: 74, Len: 73
> Data (73 bytes)

0000  02 00 00 00 45 00 00 71  29 d6 40 00 80 06 00 00   ····E··q )·@·····
0010  7f 00 00 01 7f 00 00 01  13 8d fe db e1 67 7b 3f   ·····g{?
0020  65 34 38 3f 50 18 27 f9  82 50 00 00 00 00 00 45   e48?P·'· ·P·····E
0030  02 00 07 31 32 32 30 30  37 31 00 10 6f ea 4f 77   ···12200 71··o·Ow
0040  ab e8 23 e9 f6 eb 66 b6  78 f9 7f 71 00 03 01 00   ··#···f· x··q····
0050  01 00 10 19 42 41 be bb  fa 76 53 6d 2b 14 7b d8   ····BA·· ·vSm+·{·
0060  46 41 e2 00 10 4c fc f2  31 35 2e 92 0d 37 e3 8a   FA···L·· 15.··7··
0070  fc cf 1f 94 c8                                     ·····
```

Figure 11: SERVER_HELLO message sent by server

The Wireshark capture in Figure 11 shows the server's response to the client-initiated handshake through the SERVER_HELLO message. This packet is Frame 269 and carries a TCP payload of 73 bytes transmitted from the server to the client over port 5005. Similar to the CLIENT_HELLO message, inspection of the payload reveals that the transmitted data appears as random binary values with no readable plaintext, indicating that sensitive cryptographic material is not exposed on the network.

The SERVER_HELLO message contains the server's Diffie-Hellman public value along with a digital signature generated using the server's RSA private key. Upon receiving this message, the client verifies the signature using the server's public key, confirming the server's identity and preventing impersonation attacks. The successful exchange of CLIENT_HELLO and SERVER_HELLO messages allows both parties to independently compute the same shared secret and derive a session key. Wireshark confirms that all subsequent packets contain encrypted payloads, demonstrating that secure communication is established only after mutual authentication and key agreement are completed.

- **Encrypted message exchange using session keys:**



tcp.port == 5005

No.	Time	Source	Destination	Protocol	Length	Info
197	23.607050	127.0.0.1	127.0.0.1	TCP	44	5005 → 65181 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
264	39.715647	127.0.0.1	127.0.0.1	TCP	56	65243 → 5005 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
265	39.715743	127.0.0.1	127.0.0.1	TCP	56	5005 → 65243 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
266	39.715792	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
267	39.718912	127.0.0.1	127.0.0.1	TCP	117	65243 → 5005 [PSH, ACK] Seq=1 Ack=1 Win=2619648 Len=73
268	39.718945	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=1 Ack=74 Win=2619648 Len=0
269	39.720050	127.0.0.1	127.0.0.1	TCP	117	5005 → 65243 [PSH, ACK] Seq=1 Ack=74 Win=2619648 Len=73
270	39.720071	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=74 Ack=74 Win=2619648 Len=0
383	54.002488	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=74 Ack=74 Win=2619648 Len=37
384	54.002518	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=74 Ack=111 Win=2619648 Len=0
385	54.005196	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=74 Ack=111 Win=2619648 Len=2
386	54.005266	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=111 Ack=76 Win=2619648 Len=0
403	56.045793	127.0.0.1	127.0.0.1	TCP	81	65243 → 5005 [PSH, ACK] Seq=111 Ack=76 Win=2619648 Len=37
404	56.045823	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=76 Ack=148 Win=2619648 Len=0
405	56.048740	127.0.0.1	127.0.0.1	TCP	46	5005 → 65243 [PSH, ACK] Seq=76 Ack=148 Win=2619648 Len=2
406	56.048818	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=148 Ack=78 Win=2619648 Len=0
459	62.649888	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [FIN, ACK] Seq=148 Ack=78 Win=2619648 Len=0
460	62.649968	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [ACK] Seq=78 Ack=149 Win=2619648 Len=0
461	62.650724	127.0.0.1	127.0.0.1	TCP	44	5005 → 65243 [FIN, ACK] Seq=78 Ack=149 Win=2619648 Len=0
462	62.650752	127.0.0.1	127.0.0.1	TCP	44	65243 → 5005 [ACK] Seq=149 Ack=79 Win=2619648 Len=0

```
> Frame 383: Packet, 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface \Device\NPF_Lo
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 65243, Dst Port: 5005, Seq: 74, Ack: 74, Len: 37
> Data (37 bytes)

0000  02 00 00 00 45 00 00 4d  2a 48 40 00 80 06 00 00   ····E··M *H@·····
0010  7f 00 00 01 7f 00 00 01  fe db 13 8d 65 34 38 3f   ····· ····e48?
0020  e1 67 7b 88 50 18 27 f9  a7 a9 00 00 00 00 00 21   ·g{·P·'· ·····!
0030  03 d5 0e 18 0a 2c 41 7b  e3 32 81 bd 4a 00 cd 93   ·····,A{ ·2··J···
0040  2d 08 be d0 0a 74 a9 4d  d4 cd c6 c1 e3 77 e0 58   -····t·M ·····w·X
0050  fc                                                  ·
```

Figure 12: Encrypted message exchange

The Wireshark capture shown in Figure 12 shows the encrypted exchange of application data following the successful completion of the handshake. This packet corresponds to Frame 383 and is transmitted from the client to the server over TCP port 5005 with a payload length of 37 bytes. Since this packet occurs after both parties have authenticated each other and derived a shared session key, the transmitted data is encrypted using AES-CBC with the negotiated session key.

Inspection of the packet payload reveals that the data appears as random binary values with no readable plaintext, confirming that user messages are not exposed on the network. This verifies that all application-level communication is protected using the session key established through Diffie-Hellman key exchange, ensuring confidentiality even in the presence of passive network attackers.

3. Security Guarantees

This implementation achieves the required security properties through a combination of symmetric encryption, public-key authentication, and key agreement mechanisms. Each property is validated through both protocol design and practical network analysis using Wireshark.

Confidentiality

Confidentiality is ensured by encrypting all application data using AES in CBC mode with PKCS#7 padding. After Phase II completes, a session key derived from the Diffie-Hellman shared secret is used for encryption. Network traffic captured in Wireshark shows that transmitted packets contain only ciphertext and initialization vectors, with no readable plaintext visible. This confirms that attackers monitoring the network cannot recover message contents.

Validation: Wireshark packet inspection shows random-looking binary payloads with no plaintext data.

Integrity

To validate integrity in practice, a temporary testing step was added during development in which an invalid or modified ciphertext was intentionally sent to the server.

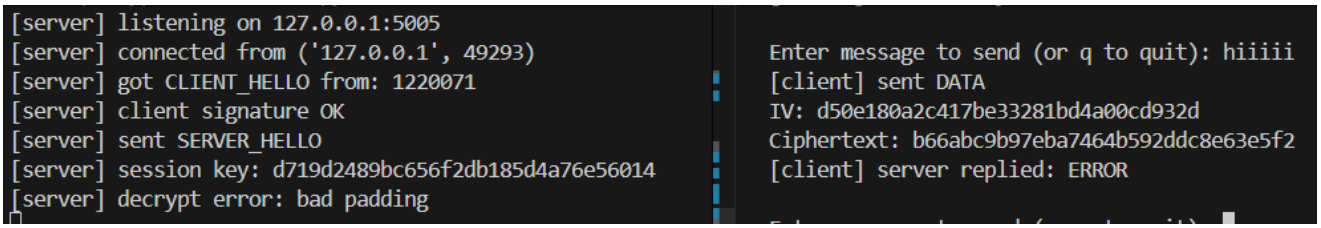


Figure 13: Integrity check

As shown in the server output, this resulted in a decryption failure due to incorrect padding ("bad padding"), and the server responded with an error message (ERROR) instead of accepting the data. This demonstrates that even small changes to the ciphertext are detected during decryption and are not silently ignored.

Because the session key is shared only after successful Diffie-Hellman key exchange and signature verification, an attacker cannot forge valid encrypted messages without knowledge of the session key. Any tampering with encrypted traffic therefore leads to decryption errors rather than successful message delivery.

Validation: When ciphertext integrity is violated, AES-CBC decryption fails with a padding error, and the server rejects the message. This confirms that message tampering is detected and integrity is preserved.

Authenticity

Authenticity is ensured through the use of RSA digital signatures during the handshake. Each participant signs their Diffie-Hellman public value using their private RSA key, and the receiver verifies the signature using the sender's public key before proceeding with the protocol.

To validate authenticity in practice, the server explicitly verifies the client's signature upon receiving the CLIENT_HELLO message. As shown in the server output previously, when the signature is correct, the handshake continues successfully.

If the signature verification fails (for example, due to an incorrect signature or mismatched public key), the server immediately terminates the handshake and raises a BAD CLIENT SIGNATURE error, as observed during testing. In this case, the connection is closed and no session key is established, preventing unauthenticated entities from participating in the protocol.

```
[server] listening on 127.0.0.1:5005
[server] connected from ('127.0.0.1', 49699)
[server] got CLIENT_HELLO from: 1220071
Traceback (most recent call last):
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\server.py", l
ine 224, in <module>
    main()
    ~~~~^^
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\server.py", l
ine 182, in main
    session_key = do_handshake(conn)
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\server.py", l
ine 149, in do_handshake
    raise ValueError("BAD CLIENT SIGNATURE")
ValueError: BAD CLIENT SIGNATURE
```

```
[client] connected
[client] sent CLIENT_HELLO
Traceback (most recent call last):
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\client.py", line 210, in <module>
    main()
    ~~~~^^
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\client.py", line 180, in main
    session_key = do_handshake(s)
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\client.py", line 152, in do_handshake
    resp = recv_message(sock)
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\client.py", line 52, in recv_message
    header = recv_exact(sock, 4)
  File "c:\Users\HP\OneDrive\Desktop\crypto\HW2\client.py", line 47, in recv_exact
    raise ConnectionError("connection closed")
ConnectionError: connection closed
```

Figure 14: Authenticity check

This behavior confirms that the server does not trust unauthenticated handshake messages and that identity verification is enforced before any secure communication takes place.

Validation: Successful RSA signature verification confirms the sender's identity, while invalid signatures cause the handshake to fail and the connection to be terminated. This demonstrates that only legitimate participants can establish a session, ensuring message authenticity.

Forward Secrecy

Forward secrecy is achieved by generating a fresh Diffie-Hellman key exchange for each session. The resulting shared secret is used to derive a temporary session key via SHA-256. Even if long-term RSA private keys are compromised in the future, previously established session keys cannot be recovered.

Validation: Each session uses a new ephemeral Diffie-Hellman key pair, producing independent session keys, as shown in the examples previously.

Conclusion

This assignment presented the design and implementation of a secure client–server communication protocol completed in two phases. Phase I ensured message confidentiality using AES-128 in CBC mode, preventing plaintext exposure over the network. Phase II addressed key distribution and authentication by integrating Diffie-Hellman key exchange with RSA digital signatures, allowing both parties to securely establish and authenticate a shared session key.

The implemented protocol achieves confidentiality, integrity, authenticity, and forward secrecy. Confidentiality is provided through encrypted application data, integrity is enforced by detecting ciphertext tampering via decryption and padding validation, and authenticity is ensured by verifying signed Diffie-Hellman values bound to student identities. Forward secrecy is achieved by using ephemeral Diffie-Hellman keys for each session.

Wireshark analysis confirmed that all handshake and data messages appear as encrypted binary payloads, and controlled tampering tests demonstrated that modified messages are correctly rejected. Overall, the implementation meets the assignment requirements and demonstrates a practical, secure communication protocol consistent with modern cryptographic design principles.